Early Detection of Silent Training Failures in Neural Networks

Sohan Bhat

November 2026

Abstract

During training, neural networks can develop defects that raise no error, so the run completes and returns a degraded model that is easy to mistake for a hard problem. This work presents LHD (Layer Health Diagnostics), a monitor that logs four signals for every layer at the end of each epoch (weight change, gradient norm, weight norm, and the fraction of dead units) and compares each one with the range it stays within during healthy runs of the same network. When a signal stays outside that range for three epochs in a row, LHD names the layer and a likely cause and stops the run. Across 645 training runs of a fully connected network, a CNN, and a small transformer, with frozen layers and dead units planted in known layers, LHD caught 411 of 470 broken runs, named the planted layer in 98% of them, and flagged 3 of 110 fresh healthy runs. Averaged within each seed, it fired 9.1 epochs before the accuracy curve (95% interval 8.4 to 9.7). A check for weights that never change caught every frozen layer two epochs sooner, so LHD helps most with dead units, where it caught 78% of faults while the fixed thresholds of existing tools either missed most of them or flagged many healthy runs.

1 Introduction

During training, neural networks can develop defects, yet nothing fails visibly since the run completes normally and simply produces a degraded result; thus, these are called "silent failures." Because of this, identifying whether the degraded result was caused by a difficult problem or a silent failure becomes difficult; therefore, it is common to misread a failure as normal. I initially encountered this as I was training my small-scale GTSRB (German Traffic Sign Recognition Benchmark) model (Stallkamp et al., 2012) from scratch, where my model plateaued at 76%, wasting weeks tuning it, until I found one convolutional layer had been accidentally excluded from the optimizer and was never properly trained. Fixing this minute detail raised the accuracy to 87%. This is not an isolated case: errors in deep learning implementations rarely cause an exception and instead reveal themselves only through degraded predictive performance (Chwilczyński et al., 2024).

Since training runs consume significant compute, every silent failure that goes undetected wastes it. The evidence existed in the form of raw numbers in the weights and biases, but perceiving that is challenging since nothing in standard practice surfaces them. To address this, this work presents LHD (Layer Health Diagnostics), a monitor that reads these numbers, catches the failure within a few epochs, identifies the broken layer and its likely cause, and halts the run before compute is wasted.

To test LHD, 645 networks were trained, and every rule and setting was fixed before the first broken run. LHD caught 411 of 470 broken runs, usually at epoch 3, while the accuracy curve caught 166 of 470, and it was earlier on average for 30 of 30 seed groups. It also flagged 3 of 110 fresh healthy runs, against 6 of 110 for the accuracy curve. However, a simple check for weights that do not change caught every frozen layer at epoch 1, two epochs before LHD, so the advantage of LHD comes from dead units. For those, it caught 78% with 3 false alarms, while the default rule of SageMaker Debugger caught 53% and flagged 12 of 110 healthy runs. Because the faults were planted to show up in the signals LHD measures, these results describe planted faults rather than bugs found in real projects.

2 Related Work

Karpathy's Recipe. In 2019, Andrej Karpathy wrote a widely cited guide for the training of neural networks where he discusses how training fails silently (Karpathy, 2019). He provides manual habits to catch these silent failures, and the Stanford CS231n course notes add checking that the update-to-weight ratio is near 0.1% (CS231n, n.d.). However, his guide simply provides a checklist a human needs to follow by hand with no mention of any automated solution.

wandb.ai. Weights & Biases (wandb) is the industry standard for experiment tracking through a dashboard where one line of code in a project can log live gradient and weight histograms during training (Biewald, 2020). Still, it is limited by only displaying the numbers and nothing more, since a human still needs to watch, understand what healthy numbers look like, and catch the anomaly.

wandb-callbacks. Wandb-callbacks is a small community add-on package for wandb that includes a dead-ReLU warning callback, which by default warns when at least 80% of a layer's units never activate (Groeger, n.d.). However, it covers only one failure mode with a single signal, provides no diagnosis or halting, and is an unmaintained hobby package rather than a supported tool.

SageMaker Debugger. Amazon SageMaker Debugger records tensors during training jobs on Amazon's cloud and checks them with built-in rules such as DeadRelu, VanishingGradient, ExplodingTensor, UnchangedTensor, and WeightUpdateRatio (Amazon Web Services, n.d.a). Any rule can be paired with an action that stops the training job when the rule fires (Amazon Web Services, n.d.b), so detection and halting are already automated. However, its dead unit, gradient, and unchanged tensor rules compare a signal against a fixed default threshold. DeadRelu, for example, fires when more than 50% of a layer's ReLUs are active less than 1% of the time, and a user training a different network has to pick a new threshold by hand. Its SimilarAcrossRuns rule compares a job with one other completed job rather than with the range seen across many healthy runs. Amazon's documentation also states that Debugger is no longer open to new customers.

AutoTrainer. AutoTrainer is a training monitor and automatic repair system published at the 2021 International Conference on Software Engineering (Zhang et al., 2021). During training, it checks gradients, weights, and the loss and accuracy curves against preset thresholds to detect five problems: vanishing gradients, exploding gradients, dying ReLU, oscillating loss, and slow convergence. When it finds one, it retrains the model with built-in fixes such as replacing the activation function, adding batch normalization, or changing the weight initializer. However, it does not report which layer is at fault (Wardat et al., 2022), and its default thresholds are the same for every network.

DeepDiagnosis. DeepDiagnosis, published at the 2022 International Conference on Software Engineering, checks a model at the end of every training epoch for eight error symptoms, including dead nodes, unchanged weights, saturated activations, and vanishing or exploding gradients (Wardat et al., 2022). When it finds a symptom, it reports the layer, maps the symptom to a likely root cause, and suggests a fix, which is close to the layer, cause, and suggested check that LHD reports. However, its checks also rely on fixed thresholds, such as counting a layer as having dead nodes when more than 70% of its ReLU units are inactive. On 188 models from the AutoTrainer benchmark with no known bugs, it still reported symptoms, including dead nodes in 21 of the 35 CIFAR-10 models (Wardat et al., 2022).

ART. Actually Robust Training is a Python library inspired by Karpathy's recipe that adds validation checks to training pipelines (Chwilczyński et al., 2024). While this is a step up, it only imposes process steps at development time rather than live fault detection from internal signals during training.

To conclude, tools that detect faults during training, and even stop or repair the run, already exist. However, their dead unit and gradient checks compare each signal against a threshold chosen in advance, such as 50%, 70%, or 80% of a layer's units being dead, even though the share of units that die during healthy training differs between networks and between layers. A threshold that suits one network can therefore miss real faults in another or flag its healthy runs, as DeepDiagnosis's dead node check did on CIFAR-10 models with no known bugs. LHD instead measures a separate healthy range for every layer, signal, and epoch from healthy runs of the network being trained. Furthermore, DeepDiagnosis is evaluated by how many buggy models it flags and how quickly, not by how many epochs earlier its signals catch a fault than the accuracy curve or by how often it flags fresh healthy runs of the same network, and those are the two measurements this work reports.

3 Method

3.1 Signals

At the end of every epoch, LHD records four signals for each layer it watches. Let W_e be the layer's weights and bias at the end of epoch e, flattened into one vector, and let $\|\cdot\|$ be the square root of the sum of squared entries. The weight change, $\|W_e - W_{e-1}\|/\|W_{e-1}\|$, measures how far the weights moved during the epoch relative to their size, so a frozen layer reads exactly 0. The weight norm, $\|W_e\|$, grows past normal before a run diverges. The gradient norm is the mean of $\|\partial L/\partial W\|$ over the epoch's batches, taken after each backward pass and before the optimizer step, and a vanishing gradient reads near 0. Finally, the dead unit fraction is the share of the layer's units whose output after the ReLU is exactly 0 for every input in a fixed probe batch of 256 validation examples. A unit is one neuron of a dense layer or one filter of a convolution, and a filter counts as dead only when it outputs 0 at every position. The probe batch is the same in every run and passes through the network in evaluation mode without gradients.

3.2 Healthy range

The healthy range comes only from calibration runs, which are trained with no defects from different random starting points. For layer ℓ, signal s, and epoch e, let $a_{\ell se}$ and $b_{\ell se}$ be the lowest and highest values across the calibration runs. A value x is out of range when

$$x < a_{\ell se}(1-m) \quad \text{or} \quad x > b_{\ell se}(1+m) + f, \tag{1}$$

with margin $m = 0.5$ and floor $f = 0.05$, and a value that is not a finite number is always out of range. The floor keeps a signal that is 0 in every healthy run, such as the dead unit fraction of most layers, from flagging a single unusual unit. Broken runs are never used to set the range, since a detector tuned on a known fault only catches that fault.

3.3 Alarm and diagnosis

LHD raises an alarm once one signal of one layer stays out of range for $p = 3$ epochs in a row. If several layers reach that point in the same epoch, it reports the first one in the order data flows through the network. The settings m, f, and p came from the LHD library's own tests and were fixed before any broken run was trained. LHD then names a likely cause from the pattern across the alarmed layer's four signals (Table 1),

Table 1: How LHD names a cause. The rules are checked from top to bottom on the layer that raised the alarm.

When the alarmed layer shows	LHD names
a signal that is not a finite number	diverged
more dead units than its range allows	dead units
weight change below its range while the gradient is inside or above	frozen layer
weight change and gradient both below their ranges	vanishing gradient
weight norm above its range	runaway weights
any other pattern	unrecognized

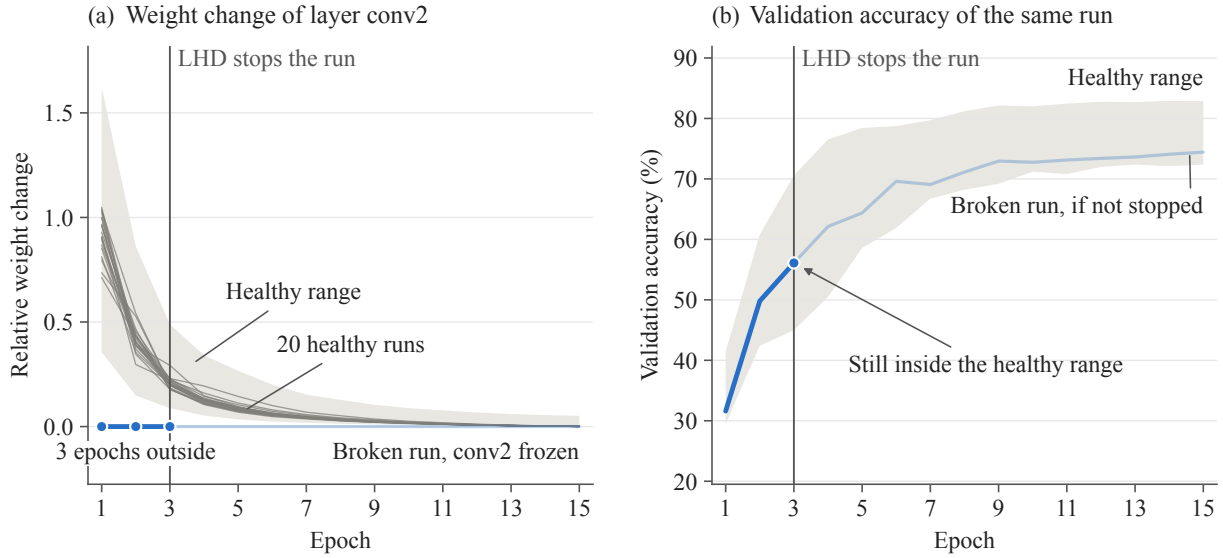


Figure 1: One CNN run with a frozen layer. (a) The frozen layer's weight change against the range of the 20 calibration runs. It stays at 0, outside the range, and LHD stops the run after three epochs. (b) Validation accuracy of the same run, which is still inside the healthy range when LHD stops it.

stops training, and reports the layer, the signals out of range, the cause, and a suggested check. For a frozen layer, for example, it suggests checking that the layer's parameters are passed to the optimizer. Figure 1 shows this on one CNN run with a frozen layer.

4 Experimental Setup

4.1 Networks and data

LHD was tested on three networks with different structures (Table 2). The fully connected network has hidden layers of 256, 128, and 64 units. The CNN has three convolutions with 8, 16, and 32 filters, each followed by a ReLU and max pooling, then a dense layer of 64 units and the output layer. The transformer predicts the next character with a width of 64, 4 attention heads, 2 blocks, and a feed-forward layer of 256 units that uses a ReLU, so dead units can be counted the same way in all three. After every epoch, validation accuracy is measured on the dataset's test split. The CNN is scored on a fixed random subset of 4,000 test images and the transformer on 512 fixed windows of 64 characters from the last 10% of the text.

Every run used Adam with a learning rate of 0.001 and a cosine schedule on one CPU thread. The seed fixes both the starting weights and the order of training batches, so a broken run and the healthy run with the

Table 2: The three networks and the runs trained for each. Calibration runs set the healthy range, and fresh healthy runs measure false alarms.

				Runs		
Network	Dataset	Parameters	Epochs	Calibration	Fresh healthy	Broken
Fully connected	Fashion-MNIST	242,762	20	20	50	135
Transformer	Tiny Shakespeare	112,577	15	20	30	170
CNN	GTSRB	41,659	15	20	30	170

same seed differ only by the fault. That healthy run is the broken run's twin, and the twin's final accuracy shows how much the fault cost.

4.2 Planted faults

Each broken run has one fault planted in one layer at the start of training. A frozen layer has its weights and bias put back after every optimizer step, so gradients are computed but never applied, which is the same failure as a layer left out of the optimizer. Dead units are made by setting the incoming weights of 10%, 25%, or 50% of a layer's units to 0 and their bias to -1, so their ReLU output is 0 for every input, and the optimizer's running averages for those units are cleared. Frozen layers were planted in every watched layer and dead units in every layer followed by a ReLU.

4.3 Runs

Calibration runs used seeds 0 to 19 and were used only to set the healthy range. Fresh healthy runs and broken runs used seeds from 1000 up, so no scored run ever touched the range. Every run trained for all of its epochs, and LHD's alarm was found by replaying the saved signals. Replaying gives the same answer as running LHD live and lets every detector be scored on the same complete runs.

4.4 Detectors compared

The accuracy curve raises suspicion when validation accuracy falls below the lowest calibration value for that epoch 3 epochs in a row. It uses the same patience as LHD and no margin, which makes it more sensitive than LHD's own rule. A version that needs only one epoch and a version based on the loss curve were also scored. Two kinds of fixed rule that need no healthy runs test whether LHD does better than checks anyone could write by hand. The zero update check fires when some layer's weights do not move at all in an epoch, like the UnchangedTensor rule of SageMaker Debugger. The dead unit checks fire when some ReLU layer has at least a fixed share of dead units, at 50% (the default of SageMaker Debugger's DeadRelu rule), 70% (DeepDiagnosis), 80% (the default of wandb-callbacks), or a single dead unit (its verbose mode). SageMaker counts a ReLU as inactive when it fires less than 1% of the time, which is close to LHD's definition but not the same.

4.5 Measures and statistics

A broken run is caught when a detector fires at or after the fault's onset, and a false alarm is any alarm on a fresh healthy run. The gap is the accuracy curve's epoch minus LHD's epoch, with "never" counted as one epoch past the end of the run. Runs that share a seed share their starting weights and batch order, and runs that share a fault and layer share the same break, so single runs are not independent. The main tests therefore average the gap within each seed and within each fault condition and apply a one sided Wilcoxon

Table 3: LHD (bold) next to the accuracy curve on the planted frozen layers and dead units. The median epoch is over the runs each method caught.

	Fully connected		Transformer		CNN	
	LHD	Accuracy	LHD	Accuracy	LHD	Accuracy
Broken runs caught	**84%**	26%	**100%**	48%	**78%**	29%
Median epoch of the alarm	**3**	17	**3**	5	**3**	7
Healthy runs flagged	**1 of 50**	1 of 50	**0 of 30**	5 of 30	**2 of 30**	0 of 30

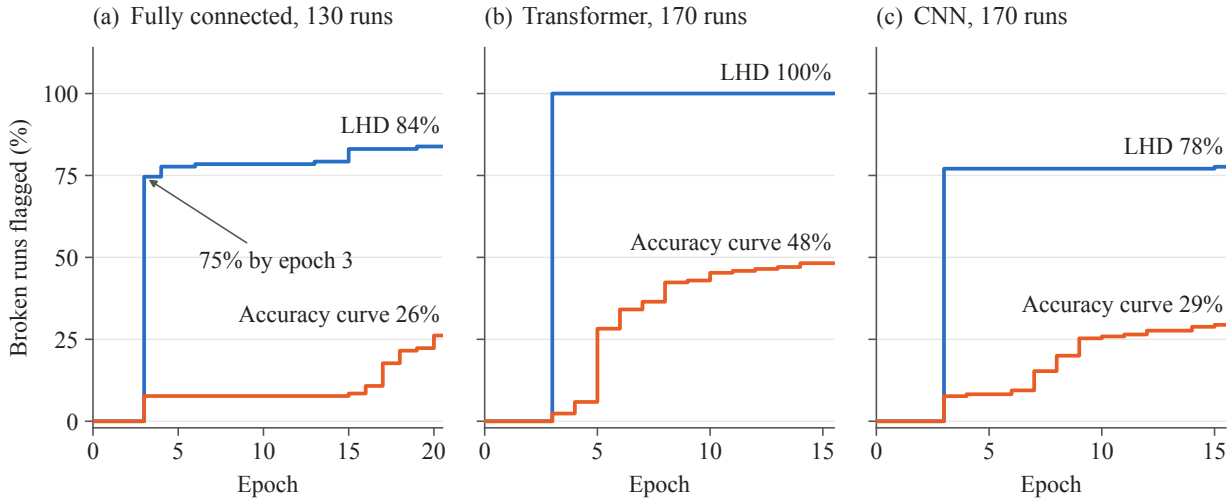


Figure 2: Share of broken runs flagged by each epoch, for LHD and the accuracy curve, in each network.

signed rank test to those averages. The 95% interval for the mean gap comes from a bootstrap with 10,000 draws that resamples whole seeds.

5 Results

5.1 Detection and timing

Across the three networks, LHD caught 411 of 470 broken runs and the accuracy curve caught 166 of 470 (Table 3). Of the runs LHD caught, the accuracy curve never noticed 61%, and 66% of all broken runs finished with a final accuracy inside the healthy range. The median alarm came at epoch 3, which is the earliest possible alarm for a fault planted at epoch 1 when three epochs out of range are required, and Figure 2 shows the share of broken runs each method had flagged by each epoch. Averaged within each seed, LHD was earlier than the accuracy curve for 30 of 30 seed groups ($p = 9.3 \times 10^{-10}$), and averaged within each fault condition it was earlier for 41 of 47 ($p < 10^{-10}$). The mean gap was 9.1 epochs, with a 95% interval of 8.4 to 9.7 from the seed bootstrap.

On fresh healthy runs, LHD raised 3 false alarms in 110 runs: one run in the fully connected network's fc2 at epoch 16, where only 1.6% of units were dead, fewer than in any calibration run (lowest 3.9%); two runs in the CNN's conv2 at epochs 3 and 4, where 6.2% of units were dead and no calibration run had a single dead unit there. When no calibration run has a dead unit in a layer, the top of its range is the floor of 0.05, so one dead filter out of 16 is enough to cross it. An alarm for too few dead units does not point to any fault, and checking the dead unit fraction against only the top of its range would remove it.

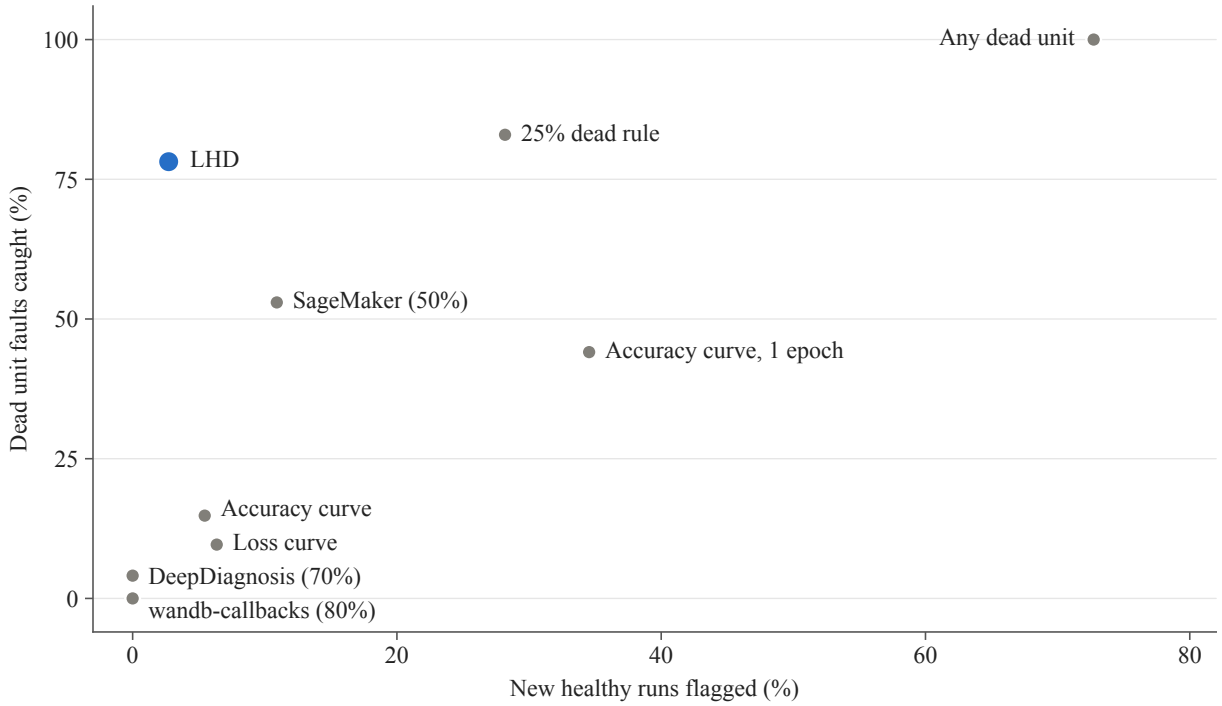


Figure 3: Dead unit faults caught against healthy runs flagged, for every detector on the same runs, pooled over the three networks. Detectors toward the upper left catch more faults while flagging fewer healthy runs.

5.2 Comparison with other detectors

For frozen layers, LHD adds nothing over the zero update check, which caught 200 of 200 at epoch 1 with no false alarms, two epochs before LHD. A frozen layer's weight change is exactly 0, so a check for an exact 0 needs neither a healthy range nor patience.

Dead units are different, because healthy networks lose units on their own and the amount varies widely. In healthy calibration runs of the CNN's dense layer, between 36% and 70% of units were dead, while no calibration run of the transformer had a single dead unit in its ReLU layers. A single fixed threshold therefore misses faults in one network or flags healthy runs in another (Figure 3). LHD caught 78% of dead unit faults with 3 false alarms in 110 healthy runs. A 25% rule came closest at 83%, but it flagged 31 healthy runs, and a rule that fires on a single dead unit caught 100% while flagging 80. The defaults of SageMaker Debugger (50%), DeepDiagnosis (70%), and wandb-callbacks (80%) caught 53%, 4%, and 0%. Some of these catches have nothing to do with the planted fault, because the same rules also fire on units that died naturally. The 50% rule caught 28 of 90 runs with 10% dead units but only 25 of 90 with 25%.

5.3 How many units must die

With 10%, 25%, and 50% of a layer's units killed, LHD caught 57%, 88%, and 90% of runs, against 7%, 9%, and 29% for the accuracy curve. Its misses came from a few layers: CNN conv3 at 10% (1 of 10 caught), CNN fc1 at 10% (0 of 10 caught), CNN fc1 at 25% (0 of 10 caught), CNN fc1 at 50% (1 of 10 caught), fully connected network fc2 at 10% (0 of 10 caught), fully connected network fc3 at 10% (0 of 10 caught), fully connected network fc3 at 25% (9 of 10 caught). In each of them, healthy runs already lose a share of units on their own, so the planted fault stayed inside the healthy range. The CNN's dense layer is the clearest case, since its healthy range already covers most of the planted values, and it shows up as the

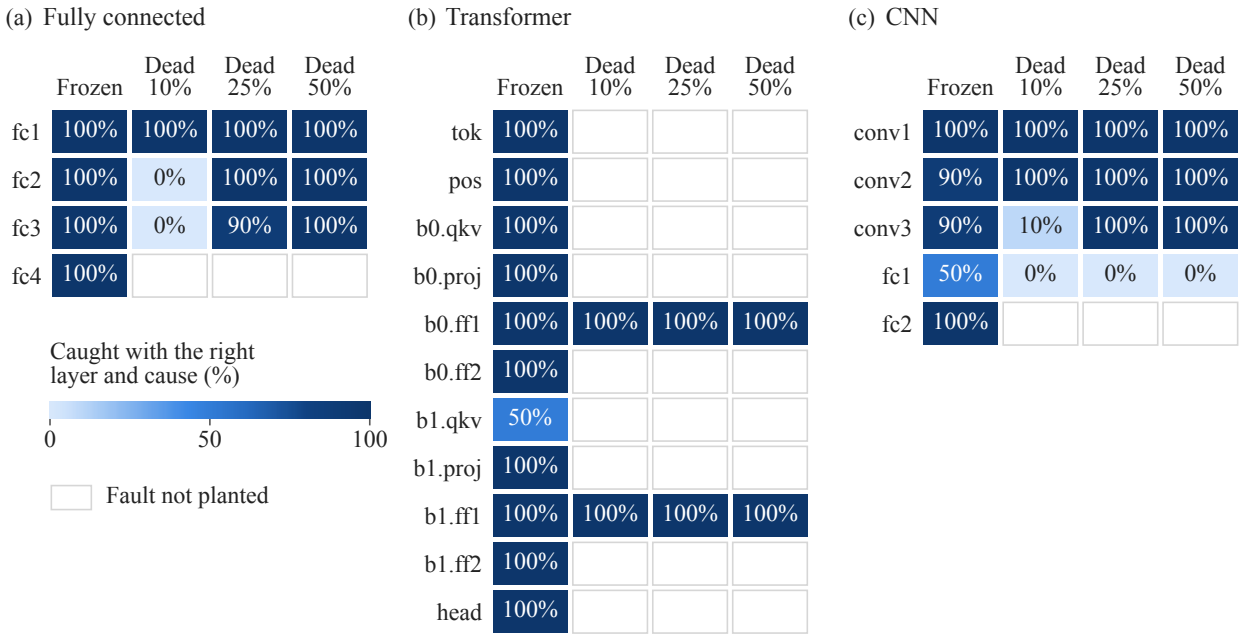


Figure 4: Share of broken runs in which LHD caught the fault and named both the planted layer and the right cause, for every layer and fault. Each cell is 10 broken runs. Blank cells are faults that were not planted, since dead units only go into layers followed by a ReLU.

pale row of Figure 4.

5.4 Faults that cost accuracy

Most planted faults did little harm. In 223 of 470 broken runs, the final accuracy was less than half a point below the healthy twin's, and the accuracy curve missing those runs is partly the curve being right. Among the 105 runs that lost 2 points or more, LHD caught 94 and the accuracy curve caught 60. Only one fault cost much on average. A frozen CNN layer lowered final accuracy by 11.1 points against the healthy twin, about as much as the frozen convolution in the GTSRB model that motivated this work, while 50% dead units in the CNN cost 3.1 points and no fault in the other two networks cost more than 0.8 points on average.

5.5 Diagnosis

When LHD caught a broken run, it named the planted layer 98.1% of the time and the planted cause 96.8% of the time (Figure 4). It named the wrong layer in 8 caught runs (50% dead units in fc1 in the CNN, reported as fc2 (1); a frozen conv2 in the CNN, reported as conv1 (1); a frozen conv3 in the CNN, reported as conv2 (1); a frozen fc1 in the CNN, reported as conv1 (1); a frozen fc1 in the CNN, reported as conv2 (1); a frozen fc1 in the CNN, reported as conv3 (3)). In each of these, the reported layer's own signal was out of range, with too many dead units in 5 and another signal in 3, so LHD found a real effect of the fault in a different layer. In the seven frozen cases, the planted layer's weight change crossed its range in the same epoch, but LHD names only the first layer in data flow order, so reporting every layer out of range would have included the planted one. It found the right layer but named the wrong cause in 5 runs (a frozen b1.qkv in the transformer, called a vanishing gradient (5)). A frozen attention projection also lowers the gradient that reaches it, so its pattern matches a vanishing gradient. The high rate of naming the right cause is largely built into the design, since the planted faults produce exactly the signal patterns the rules look for.

Table 4: What happens when signals are removed or used alone, pooled over the three networks. Changes are in percentage points against all four signals.

Signals used	Broken runs caught		Right layer and cause	
	Rate	Change	Rate	Change
All four	**87%**		**85%**	
Without weight change	70%	−18	45%	−40
Without weight norm	87%	same	85%	same
Without grad norm	87%	same	86%	+1
Without dead unit fraction	43%	−45	41%	−44
Only weight change	43%	−45	43%	−42
Only weight norm	10%	−77	0%	−85
Only grad norm	25%	−63	0%	−85
Only dead unit fraction	52%	−36	45%	−40

5.6 What each signal adds

Table 4 shows the same runs replayed with each signal removed, or with one signal used alone. Without the dead unit fraction, LHD caught 43% of broken runs instead of 87%, so that signal carries most of the detection. Without the weight change, it named the right layer and cause in 45% instead of 85%, since the weight change is what separates a frozen layer from other faults. Removing the weight norm or the gradient norm changed little, because none of the planted faults was designed to move them.

5.7 Settings and calibration

Thresholds placed right at the edge of the healthy range flagged 69% of healthy runs. Widening them by the margin of 0.5 used here cut that to 3%, while the share of broken runs caught fell only from 98% to 87%. The range also needs enough calibration runs. Built from 2 runs, it flagged up to 22% of healthy runs (in the CNN), and built from all 20, at most 7%.

5.8 Cost

Monitoring added 11.6% for the fully connected network, 5.8% for the transformer, and 2.2% for the CNN to the training time, measured on fresh healthy runs on one CPU thread. Most of that time goes to the gradient norm, read after every batch, and to the probe batch, run once per epoch, so the overhead is largest for the network with the cheapest epochs. Stopping each broken run at LHD's alarm would have saved 70% of its training on average, and all runs together took 94 hours of CPU time on one laptop.

6 Discussion and Limitations

The results support the hypothesis for planted faults. Per layer signals exposed a broken layer within the first few epochs, usually before the accuracy curve showed anything, and LHD rarely flagged healthy runs. A frozen layer, however, is caught just as well and sooner by a check for weights that never change, so for that fault the only gain is that the same rules also cover it. For dead units, a range measured on healthy runs of the same network did better than every fixed threshold tested, including the defaults of three existing tools.

The faults were planted to show up in the signals LHD measures, so the detection and diagnosis rates describe how well LHD separates these faults from normal variation, not how often it would catch bugs in

real projects. Many planted faults also cost less than one point of accuracy, which means stopping those runs saves compute on runs that would have produced a nearly normal model. LHD misses dead units in layers where healthy runs already lose many units, because the healthy range treats that loss as normal. Building the range takes about 10 to 20 healthy runs of the same network, which costs more than LHD can save for a model trained only once, and it assumes those runs are themselves healthy. Finally, the networks are small and were trained for 15 to 20 epochs on a laptop CPU, the transformer reaches only about 37% next character accuracy, and large networks, long runs, other optimizers, and GPU training were not tested.

7 Conclusion

This work presented LHD, a monitor that compares four signals of every layer with ranges measured on healthy runs of the same network and stops a run when one stays out of range. On 645 runs across three networks, it caught 87% of planted frozen layers and dead units, usually within three epochs, named the right layer in 98% of those runs, and flagged 3 of 110 healthy runs. Its advantage over existing tools is clearest for dead units, where a healthy range for each layer avoided both the misses and the false alarms of fixed thresholds. The next step is to test LHD on real bugs collected from training code.

Code and data

The code, every saved run, and the tables behind each number are at https://github.com/sohan-bhat/lhd.

References

Amazon Web Services (n.d.a). List of Debugger built-in rules. Amazon SageMaker AI Developer Guide. https://docs.aws.amazon.com/sagemaker/latest/dg/debugger-built-in-rules.html

Amazon Web Services (n.d.b). Use Debugger built-in actions for rules. Amazon SageMaker AI Developer Guide. https://docs.aws.amazon.com/sagemaker/latest/dg/debugger-built-in-actions.html

Biewald, L. (2020). Experiment tracking with Weights and Biases. Software available from wandb.com. https://www.wandb.com/

Chwilczyński, S., Trębacz, K., Cyganik, K., Małecki, M., and Brzezinski, D. (2024). ART: Actually Robust Training. In *Machine Learning and Knowledge Discovery in Databases. Research Track and Demo Track (ECML PKDD 2024)*, Lecture Notes in Computer Science, vol. 14948, pp. 374–378. Springer, Cham. https://doi.org/10.1007/978-3-031-70371-3_23. Preprint: arXiv:2408.16285.

CS231n (n.d.). Neural networks part 3: Learning and evaluation. Course notes for CS231n, Stanford University. https://cs231n.github.io/neural-networks-3/

Groeger, F. (n.d.). wandb-callbacks: Additional callbacks for Weights & Biases. GitHub repository. https://github.com/FabianGroeger96/wandb-callbacks

Karpathy, A. (2019). A recipe for training neural networks. Blog post, April 25, 2019. https://karpathy.github.io/2019/04/25/recipe/

Stallkamp, J., Schlipsing, M., Salmen, J., and Igel, C. (2012). Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32:323–332. https://doi.org/10.1016/j.neunet.2012.02.016

Wardat, M., Cruz, B. D., Le, W., and Rajan, H. (2022). DeepDiagnosis: Automatically diagnosing faults and recommending actionable fixes in deep learning programs. In *Proceedings of the 44th International Conference on Software Engineering (ICSE 2022)*, pp. 561–572. ACM, New York. https://doi.org/10.1145/3510003.3510071. Preprint: arXiv:2112.04036.

Zhang, X., Zhai, J., Ma, S., and Shen, C. (2021). AUTOTRAINER: An automatic DNN training problem detection and repair system. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 359–371. IEEE. https://doi.org/10.1109/ICSE43902.2021.00043